

[Algorithm]

분할정복알고리즘 : 합병 정렬 알고리즘

인하대학교 소프트웨어융합공학과

김종훈 교수



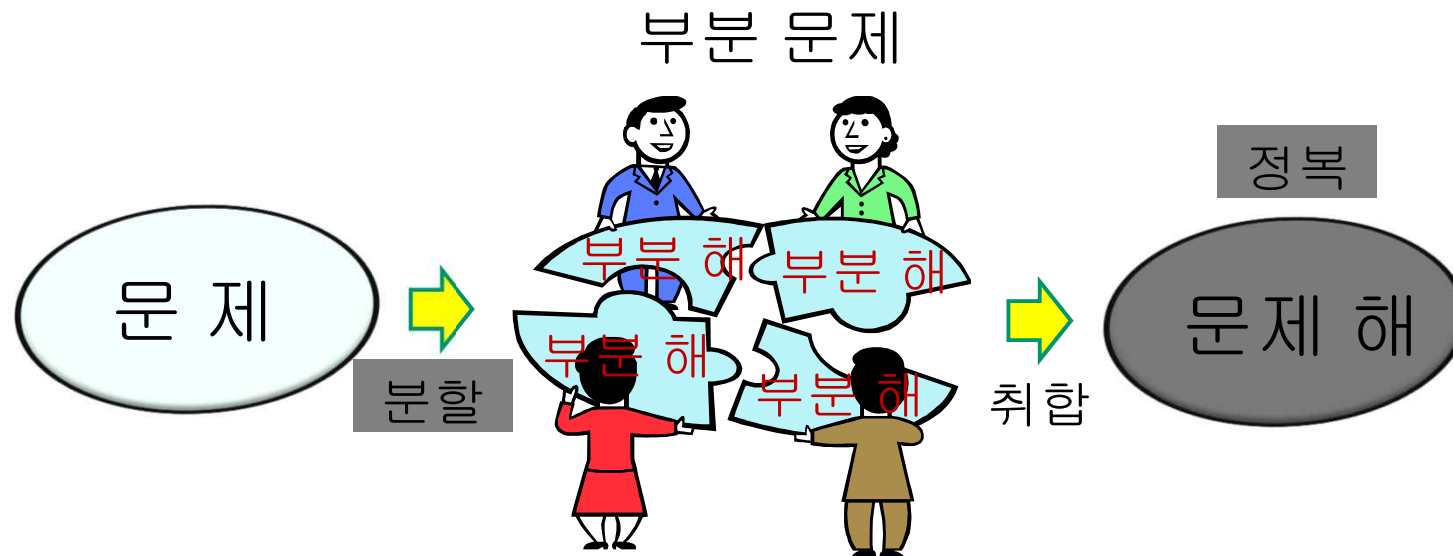
인하대학교
INHA UNIVERSITY

분할 정복 알고리즘

- 주어진 문제의 입력을 분할하여 문제를 해결(정복)하는 방식의 알고리즘
 - 분할한 입력에 대하여 동일한 알고리즘을 적용하여 해를 계산
 - 이들의 해를 취합하여 원래 문제의 해를 얻음

- 부분문제와 부분해
 - 분할된 입력에 대한 문제를 부분문제 (subproblem)
 - 부분문제의 해를 부분해
 - 부분문제는 더 이상 분할할 수 없을 때까지 계속 분할

분할 정복 알고리즘



분할

- 크기가 n 인 입력을 2개로 분할하고, 각각 분할된 부분 문제의 크기가 $n/2$ 일 경우의 분할 예

입력 크기

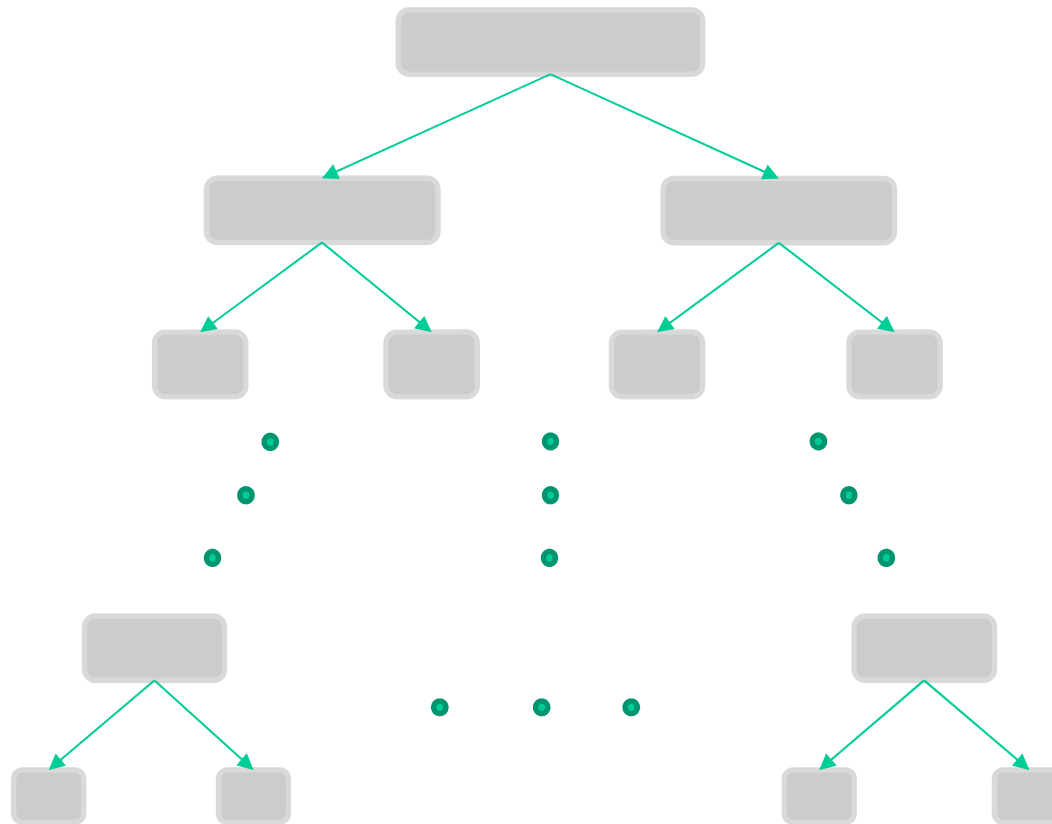
n

$\frac{n}{2}$

$\frac{n}{2^2}$

$\frac{n}{2^{k-1}}$

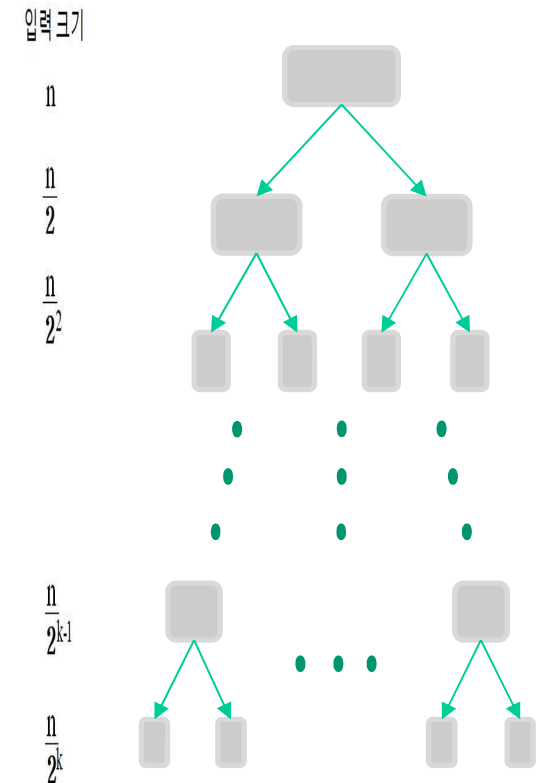
$\frac{n}{2^k}$



분할

□ 입력 크기가 n 일 때 총 분할 횟수

- 총 분할한 횟수 = k 라고 놓는다.
- 1번 분할 후 각각의 입력 크기 $n/2$
- 2 번 분할 후 각각의 입력 크기 $n/2^2$
-
- k 번 분할 후 각각의 입력 크기 $n/2^k$
- 따라서 $n/2^k = 1$ 일 때 더 이상 분할할 수 없다.
- $k = \log_2 n$



정복 과정

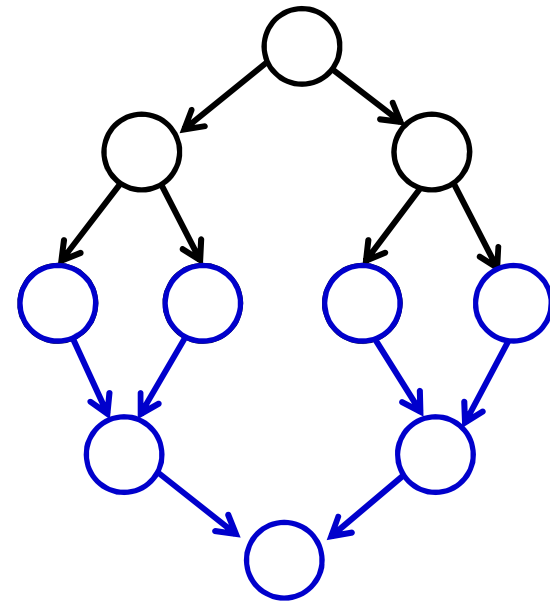
□ 대부분의 분할 정복 알고리즘

- 문제의 입력을 단순히 분할만 해서는 해를 구할 수 없다.

□ 따라서 분할된 부분 문제들을 정복해야 함

- 부분해를 찾아야 한다.
- 정복하는 방법은 문제에 따라 다르다.
- 일반적으로 부분 문제들의 해를 취합하여 보다 큰 부분 문제의 해를 구한다.

분할 과정



정복 (취합) 과정

분할 정복 알고리즘의 분류

- 분할 정복 알고리즘은 분할되는 부분문제의 수와 부분문제의 크기에 따라서 다음과 같이 분류
 - 문제가 a 개로 분할되고, 부분문제의 크기가 $1/b$ 로 감소하는 알고리즘
 - 합병 정렬
 - 문제가 2개로 분할되고, 부분문제의 크기가 일정하지 않은 크기로 감소하는 알고리즘
 - 퀵 정렬
 - 문제가 2개로 분할되나, 그 중에 1개의 부분문제는 고려할 필요가 없으며, 부분문제의 크기가 $1/2$ 로 감소하는 알고리즘
 - 이진탐색
 - 문제가 2개로 분할되나, 그 중에 1개의 부분문제는 고려할 필요가 없으며, 부분문제의 크기가 일정하지 않은 크기로 감소하는 알고리즘
 - 선택 문제 알고리즘
 - 부분문제의 크기가 1, 2개씩 감소하는 알고리즘
 - 삽입 정렬

합병 정렬 (Merge Sort)

- 합병 정렬은 입력이 2개의 부분문제로 분할되고, 부분문제의 크기가 $1/2$ 로 감소하는 분할 정복 알고리즘
 - n 개의 숫자들을 $n/2$ 개씩 2개의 부분문제로 분할
 - 각각의 부분문제를 재귀적으로 합병 정렬
 - 2개의 정렬된 부분을 합병하여 정렬(정복)

- 합병 과정이 문제를 정복하는 것

합병 (merge)

- 2개의 각각 정렬된 숫자들을 1개의 정렬된 숫자로 합치는 것

배열 A: 6 14 18 20 29

배열 B: 1 2 15 25 30 45

⇒ 배열 C: 1 2 6 14 15 18 20 25 29 30 45

출력: 정렬된 $A[p] \sim A[q]$

- ```

1. if (p < q) { // 배열의 원소의 수가 2개 이상이면
2. k = [(p+q)/2]
 // k=반으로 나누기 위한 중간 원소의 인덱스
3. MergeSort(A,p,k) // 앞부분 재귀 호출
4. MergeSort(A,k+1,q) // 뒷부분 재귀 호출
5. A[p]~A[k]와 A[k+1]~A[q]를 합병한다.
 }

```

# 합병 정렬 알고리즘

## □ Line 1

- 정렬할 부분의 원소의 수가 2개 이상일 때에만 다음 단계 수행. 만일  $n=1$ 이면, 그 자체로 정렬된 것이므로 어떤 수행을 할 필요 없이 이전 호출했던 곳으로 리턴

## □ Line 2

- 정렬할 부분의 원소들을  $\frac{1}{2}$ 로 나누기 위해,  $k = \lfloor (p+q)/2 \rfloor$ 를 계산. 즉, 원소의 수가 홀수인 경우에는  $k$ 는 소수점 이하를 버림

## □ Line 3~4

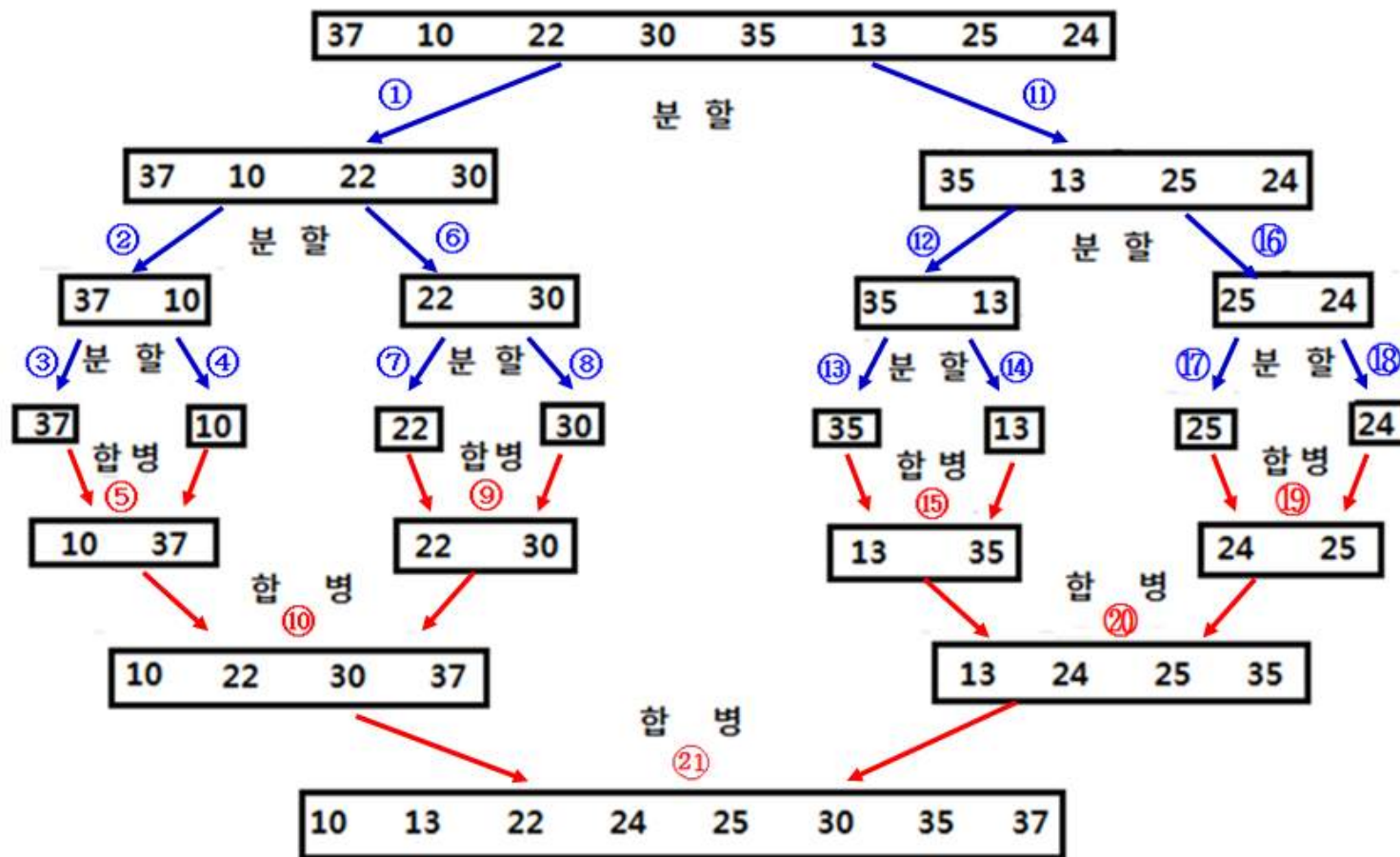
- MergeSort(A,p,k)와 MergeSort(A,k+1,q)를 재귀 호출하여 각각 정렬

## □ Line 5

- line 3~4에서 각각 정렬된 부분을 합병
- 합병 과정의 마지막에는 임시 배열에 있는 합병된 원소들을 배열 A로 복사. 즉, 임시 배열 B[p]~B[q]를 A[p]~A[q]로 복사

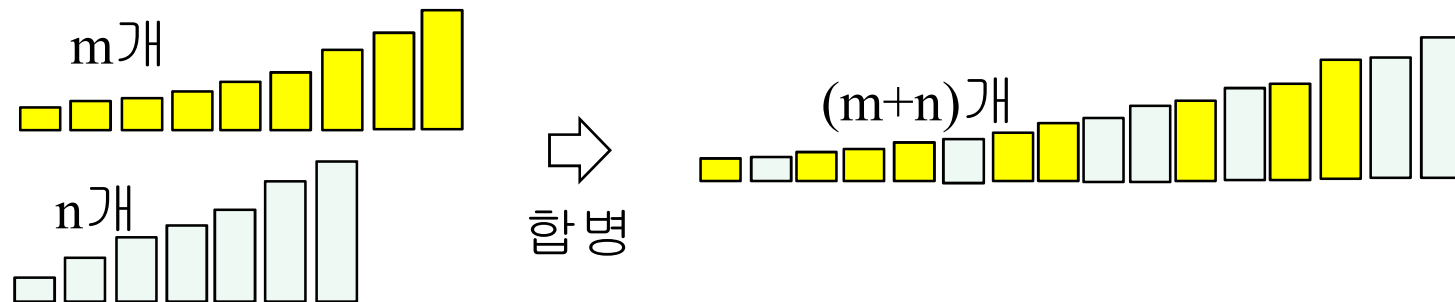
# 합병 정렬 알고리즘의 수행 과정

□ n=8인 배열 A=[37, 10, 22, 30, 35, 13, 25, 24]



# 시간복잡도

- 분할하는 부분은 배열의 중간 인덱스 계산과 2번의 재귀 호출이므로  $O(1)$  시간 소요
- 합병의 수행 시간은 입력의 크기에 비례.
  - 2개의 정렬된 배열 A와 B의 크기가 각각 m과 n이라면, 최대 비교 횟수 =  $(m+n-1)$
  - 합병의 시간복잡도 =  $O(m+n)$

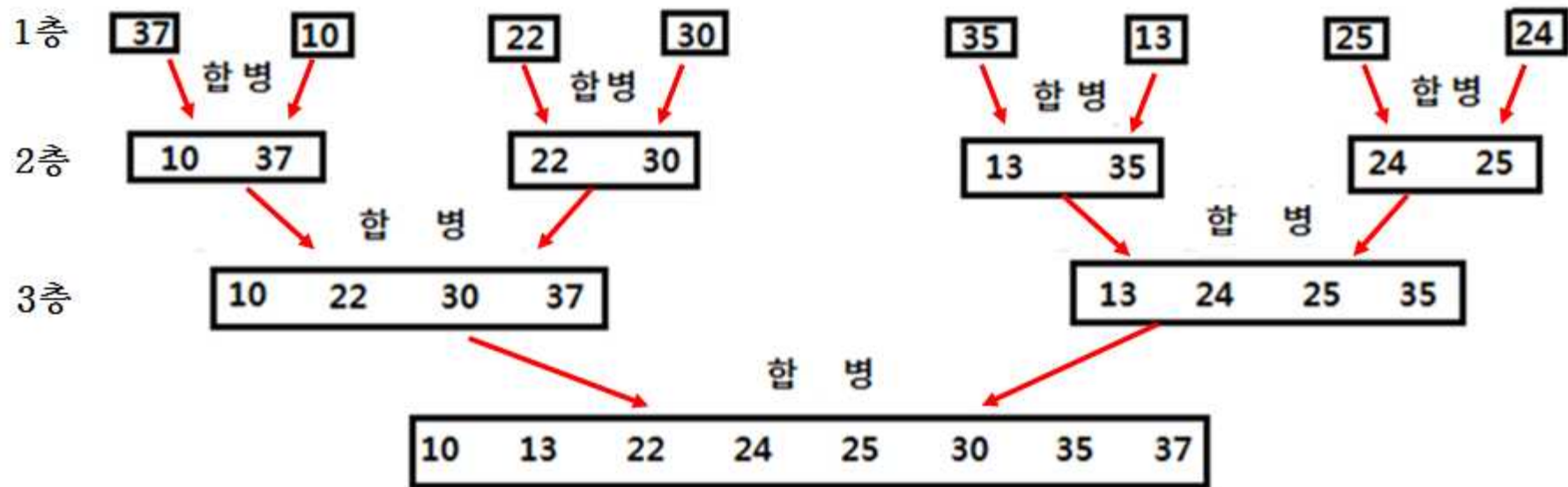


- 합병 정렬에서 수행되는 총 비교 횟수
  - 각각의 합병에 대해서 몇 번의 비교가 수행되었는지를 계산하여 이들을 모두 합한 수

# 시간복잡도

## □ 합병별 비교횟수

- 각 층을 살펴보면 모든 숫자(즉,  $n=8$ 개의 숫자)가 합병에 참여
- 합병은 입력 크기에 비례하므로 각 층에서 수행된 비교 횟수는  $O(n)$



# 시간복잡도

## □ 층수의 계산

- 층수를 세어보면, 8개의 숫자를 반으로, 반의 반으로 반의 반의 반으로 나눈다.
- 이 과정을 통하여 3층이 만들어진다.

| 입력 크기         | 예 | 층  |
|---------------|---|----|
| $n$           | 8 |    |
| $n/2$         | 4 | 1층 |
| $n/4 = n/2^2$ | 2 | 2층 |
| $n/8 = n/2^3$ | 1 | 3층 |

# 시간복잡도

## □ 입력의 크기가 $n$ 일 때 몇 개의 층이 만들어질까?

- $n$ 을 계속하여  $1/2$ 로 나누다가, 더 이상 나눌 수 없는 크기인  $1$ 이 될 때 분할을 중단한다.
- 따라서  $k$ 번  $1/2$ 로 분할했으면  $k$ 개의 층이 생기는 것이고,  $k$ 는  $2^k=n$ 으로 부터  $\log_2 n$ 임을 알 수 있다.

## □ 합병 정렬의 시간복잡도:

- (층수)  $\times O(n) = \log_2 n \times O(n) = O(n \log n)$



# 합병 정렬의 단점

- 대부분의 정렬 알고리즘들은 입력을 위한 메모리 공간과  $O(1)$  크기의 메모리 공간만을 사용하면서 정렬 수행
  - $O(1)$  크기의 메모리 공간이란 입력 크기  $n$ 과 상관없는 크기의 공간(예를 들어, 변수, 인덱스 등)을 의미
- 합병 정렬의 공간 복잡도:  $O(n)$ 
  - 입력을 위한 메모리 공간 (입력 배열)외에 추가로 입력과 같은 크기의 공간 (임시 배열)이 별도로 필요.
  - 2개의 정렬된 부분을 하나로 합병하기 위해, 합병된 결과를 저장할 곳이 필요하기 때문

# 응용

- 합병정렬의 복잡도는  $O(n \log n)$ 으로 선택 정렬이나 삽입 정렬의 계산 복잡도  $O(n^2)$ 보다 낮아 정렬해야하는 자료의 개수가 많을수록 훨씬 더 빠른 정렬 성능을 발휘
- 멀티코어 (Multi-Core) CPU와 다수의 프로세서로 구성된 그래픽 처리 장치 (Graphic Processing Unit)의 등장으로 정렬 알고리즘을 병렬화하는 데에 합병 정렬 알고리즘이 활용



**Thank You**